Operator-Level Boolean Computation with Correspondence Matrices

Brian Theory

Abstract

Correspondence matrices (CMs) encode binary logical connectives as 2×2 Boolean operators in a declared operand frame. Their purpose is not merely to reshape the four values of a truth table. Once the operand encoding and scalar algebra are fixed, a CM can be evaluated, transformed to account for operand order and polarity, and fused with other operator matrices before the operands themselves are evaluated. We define CM evaluation as an XOR–AND contraction over GF(2): conjunction replaces arithmetic multiplication and exclusive-or, written $\Updownarrow$, replaces arithmetic addition. True-first one-hot bras and kets make this contraction select exactly the connective value associated with the two operands. We then distinguish linearity in the operator coefficients from linearity of a Boolean function in its inputs, derive the row, column, and transpose transformations induced by signed operand changes, and state the condition under which pointwise operator fusion is sound. A formula-valued logical matrix (LM) provides a symbolic lift: positive valuation produces the corresponding numeric CM, and valuation commutes with Boolean operations and the LM pairing. This separates LMs from algebraic-normal-form coefficients and from semi-tensor-product logical structure matrices. Together, these results provide the typed foundation for a compiler method that normalizes compatible operand frames and performs verified operator-level fusion. The empirical part of the study is specified as a frozen evaluation of when this rewrite saves total work after compilation, memory, and interface costs are included; no performance result is claimed before that campaign is completed.

1 Introduction

Boolean computation is usually written as a sequence of operations on truth values or as a symbolic expression whose internal nodes are logical connectives. Either view makes the operands prominent and leaves each connective as a small instruction attached to an expression node. A correspondence matrix reverses that emphasis. It packages a binary connective as a 2×2 operator whose rows and columns are indexed by the states of its two operands. Once those axes are declared consistently, the operator can be evaluated, transformed, and combined as an object in its own right.

This operator interpretation is the essential point of the construction. A CM contains the same four output bits as a binary truth table, so its information content is not new. What changes is how those bits participate in computation. A true-first bra for the left operand, a CM in the middle, and a true-first ket for the right operand form a Boolean contraction. Because both states are one-hot, the contraction selects exactly one matrix entry. The matrix therefore acts as the connective rather than merely displaying it.

The same view becomes useful before evaluation. Transposition exchanges the operands, row or column permutations absorb input negations, and entrywise complement negates the output. These transformations can place two operator occurrences into an identical *operand frame*: the same variables, axis order, polarity, and truth-state convention. When the frames agree, an outer Boolean connective can be applied entrywise to the two CMs. The resulting CM replaces the two subexpressions without changing their value. The compiler question is therefore not whether a truth

table can be reshaped, but whether operator normalization and fusion can remove repeated work often enough to repay their preparation and representation costs.

Matrix treatments of logic have a substantial history. Boolean matrix algebras were developed for computer manipulation of logical functions well before CMs [Edwards, 1972]. Matrix logic and vector logic treat logical connectives as operators, including outer-product operator bases and matrices acting on Kronecker products of truth vectors [Stern, 1988, Mizraji, 1992, 2008]. Semi-tensor-product work gives numeric logical structure matrices for Boolean functions and network dynamics [Cheng and Qi, 2010, Zhao et al., n.d.], while Cheng et al. [2011] explicitly uses Boolean matrix operations with XOR as addition and AND as multiplication. Accordingly, this paper does not claim that truth tables, Boolean matrix products, logical matrix operators, or pointwise combination of truth values are new.

The contribution sought here is narrower and computationally testable: a typed pipeline that represents a binary connective as a compact CM, records its operand frame, aligns signed variable pairs through explicit matrix transformations, fuses only exact compatible frames, and exposes a fallback when structural fusion is unavailable. The mathematical identities are elementary or antecedented; their role is to make the compiler transformation precise enough to prove and measure. The paper makes the following contributions.

1. It defines CM evaluation as XOR–AND contraction over GF(2) and separates this algebra from ordinary real or integer matrix multiplication.

2. It gives a typed operand-frame calculus for exchange, input polarity, output negation, and sound same-operands fusion.

3. It separates and proves the structural alignment/fusion rule, exact four-assignment retabulation, and the top-level admitted compiler path.

4. It defines formula-valued LMs and proves that valuation produces a CM in the corresponding frame and commutes with logical pairing.

5. It separates LMs and CMs from external ANF and STP coordinate representations, preventing several superficially similar objects from being treated as the same mathematical type.

6. It specifies an empirical boundary question: when do normalization and fusion reduce total work after preparation, compilation, memory, kernel, and interface costs are all counted?

An earlier public manuscript introduced Correspondence Matrices under the author's former name [Droncheff, 2018]. The present work is a substantial revision rather than a republication of that text. It narrows the novelty claim, makes the scalar algebra and basis order explicit, replaces the earlier higher-dimensional size formulas with the correct 2^n entry count, removes the proposed equivalence with ordinary modulo, and treats measurement and superposition only as nonphysical Boolean operations. It also adds the operand-alignment compiler rule, a versioned correctness argument, and a new evaluation designed to report positive, neutral, and negative performance regions.

The remainder of the paper first defines CM operators and their evaluation semantics, then presents operand alignment, fusion, and the compiler invariant before the denser symbolic material. It next compares the construction with matrix logic, STP, ANF, and small-function logic synthesis, develops the formula-valued LM as a secondary symbolic extension, and specifies the empirical boundary. Most formal proofs and exhaustive finite checks are placed in the appendix and reproducibility supplement.

2 Correspondence matrices as Boolean operators

2.1 States, axes, and scalar algebra

Let $\mathbb{B} = \{0, 1\}$. We write conjunction as $\wedge$, negation as $\neg$, and exclusive-or as $\Updownarrow$.[1] When bits are regarded as elements of GF(2), multiplication is conjunction and addition is exclusive-or. Every displayed CM contraction uses this algebra. It is not an arithmetic matrix product over the integers, reals, or complex numbers.

For $x \in \mathbb{B}$, define the true-first state

$$|x\rangle = \begin{bmatrix} x \\ \neg x \end{bmatrix}, \qquad \langle x| = \begin{bmatrix} x & \neg x \end{bmatrix}. \tag{1}$$

The first component represents truth and the second represents falsity. CM rows are ordered by the left operand $X = 1, 0$, and columns by the right operand $Y = 1, 0$. This declared order is part of the type of the representation; a software array using another order must be converted explicitly.

For a binary operator $\Theta : \mathbb{B}^2 \to \mathbb{B}$, define

$$[\Theta] := \begin{bmatrix} \Theta_{11} & \Theta_{10} \\ \Theta_{01} & \Theta_{00} \end{bmatrix} = \begin{bmatrix} 1\,\Theta\,1 & 1\,\Theta\,0 \\ 0\,\Theta\,1 & 0\,\Theta\,0 \end{bmatrix}. \tag{2}$$

Thus $\Theta_{xy} := x\,\Theta\,y$ denotes one matrix entry. The bare bracketed operator, such as $[\Updownarrow]$, names a particular CM; $[\Theta]$ is the generic form. There is no function subscript on the whole matrix in the main-text notation because the operator itself already supplies the needed identity.

2.2 XOR–AND contraction

For any $C \in \mathbb{B}^{2\times 2}$, define $\chi_x(r) = 1$ when $r = x$ and $\chi_x(r) = 0$ otherwise. The row and column state encodings have this same component function. Then

$$E_C(x, y) := \langle x|\, C\, |y\rangle := \bigdownarrow_{r,s\in\{1,0\}} \chi_x(r) \wedge C_{rs} \wedge \chi_y(s). \tag{3}$$

Equation (3) is a Boolean contraction: AND forms each term and XOR reduces the four terms. The compact bra–operator–ket notation suppresses those mechanics after they have been declared.

Proposition 1 (CM selection)**.** *For every binary Boolean operator Θ and $x, y \in \mathbb{B}$,*

$$\langle x|\, [\Theta]\, |y\rangle = x\,\Theta\,y. \tag{4}$$

Proof. Each state in (1) has exactly one nonzero component. In the four-term contraction, every term except the one indexed by (x, y) therefore contains a zero factor. The surviving term is $1 \wedge \Theta_{xy} \wedge 1 = \Theta_{xy}$. □

For XOR, the operator matrix is

$$[\Updownarrow] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{5}$$

[1] The paper uses $\Updownarrow$ (`\Updownarrow`) rather than a circled-plus glyph. In the declared true-first frame, the XOR CM is a 90° rotation and the entrywise complement of the XNOR CM. Rotating the XNOR glyph visually gives the chosen XOR glyph, and overlaying the two motivates the Impax glyph used below.

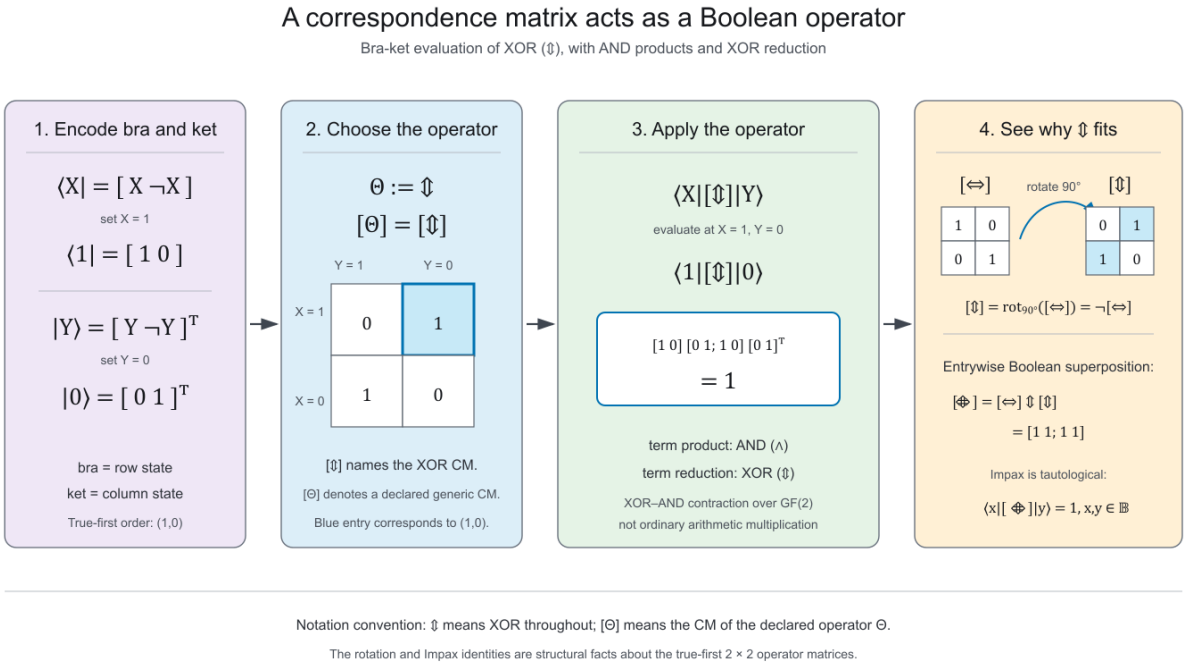


Figure 1: A binary truth value is encoded as a true-first bra or ket, and $[\Theta]$ acts between them through XOR–AND contraction. The XOR CM $[⇳]$ is a 90° rotation and entrywise complement of the XNOR CM $[⇔]$. Their entrywise Boolean superposition is the tautological Impax CM $[⟴]$.

Taking $x = 1$ and $y = 0$ gives

$$\langle 1| \, [⇳] \, |0\rangle = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1, \tag{6}$$

where the displayed products and reduction retain the AND/XOR meaning of (3). Figure 1 presents the same evaluation visually and introduces the geometric notation used below.

2.3 Coefficient-space linearity

The contraction is linear in the four operator coefficients over GF(2). For fixed x and y and matrices $A, B \in \mathbb{B}^{2\times 2}$,

$$E_{A⇳B}(x, y) = E_A(x, y) ⇳ E_B(x, y), \qquad E_0(x, y) = 0, \tag{7}$$

where $A ⇳ B$ is entrywise XOR. This statement is deliberately narrow. It does not say that the Boolean function represented by a CM is linear in x and y. Nor does it make the state encoding linear: the map $x \mapsto [x, \neg x]^{\mathsf{T}}$ is affine because it sends 0 to $[0, 1]^{\mathsf{T}}$ rather than the zero vector. If the contraction is extended to arbitrary vectors in $\mathrm{GF}(2)^2$, it is separately linear in the bra, the matrix coefficients, and the ket; that multilinearity remains distinct from the algebraic degree of the represented Boolean function.

Let E_{rs} contain a one at position (r, s) and zeros elsewhere. The four matrices $E_{11}, E_{10}, E_{01}, E_{00}$ form the standard basis of $\mathrm{GF}(2)^{2\times 2}$, and

$$[\Theta] = \bigdoublebarwedge_{r,s\in\{1,0\}} \Theta_{rs} E_{rs}. \tag{8}$$

Equation (4) extracts the corresponding coefficient. This is a minterm-indicator or operator-basis expansion, not the algebraic normal form of Θ in uncomplemented variables.

2.4 XNOR, XOR, and Impax

The true-first XNOR and XOR matrices are

$$[\Leftrightarrow] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad [\Updownarrow] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{9}$$

Consequently, define the clockwise rotation of a displayed 2×2 array by

$$\mathrm{rot}_{90^\circ} \begin{bmatrix} a & b \\ c & d \end{bmatrix} := \begin{bmatrix} c & a \\ d & b \end{bmatrix}.$$

For this particular pair of operators,

$$[\Updownarrow] = \mathrm{rot}_{90^\circ}([\Leftrightarrow]) = \neg[\Leftrightarrow]. \tag{10}$$

The Impax operator is the tautological operator represented by the centered overlay glyph $\Circled{$\Updownarrow$}$. Entrywise Boolean XOR gives

$$[\Circled{$\Updownarrow$}] = [\Leftrightarrow] \Updownarrow [\Updownarrow] = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \langle x | \, [\Circled{$\Updownarrow$}] \, | y \rangle = 1 \quad (x, y \in \mathbb{B}). \tag{11}$$

Here *superposition* refers only to the entrywise Boolean operation in (11); it does not denote a quantum state.

3 Operand transformations and aligned fusion

A CM occurrence is typed by an operand frame: its row variable, column variable, input polarities, and truth-state order. Let $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. With the true-first axes of (2), operand exchange, input negation, and output negation act as

$$[\Theta(Y, X)] = [\Theta]^{\mathsf{T}}, \qquad [\Theta(\neg X, Y)] = P[\Theta], \tag{12}$$

$$[\Theta(X, \neg Y)] = [\Theta]P, \qquad [\neg\Theta] = \mathbf{1} \Updownarrow [\Theta]. \tag{13}$$

The matrix products involving P are XOR–AND products; here they simply permute rows or columns. They are not a general NPN canonicalizer. Their role is to rewrite an admitted signed pair into one explicitly declared operand frame.

For a binary outer operator Φ, define entrywise fusion by

$$([\Theta_1] \, \Phi \, [\Theta_2])_{rs} := (\Theta_1)_{rs} \, \Phi \, (\Theta_2)_{rs}. \tag{14}$$

If both matrices act on the same ordered operands in the same frame, then

$$(X \, \Theta_1 \, Y) \, \Phi \, (X \, \Theta_2 \, Y) = \langle X | \, ([\Theta_1] \, \Phi \, [\Theta_2]) \, | Y \rangle . \tag{15}$$

Both sides select position (X, Y), so the identity holds for every binary Φ without requiring Φ to distribute over XOR. Pointwise truth-table combination is established prior art; the compiler-specific claim is the typed admission rule that first proves frame agreement and otherwise refuses this fusion.

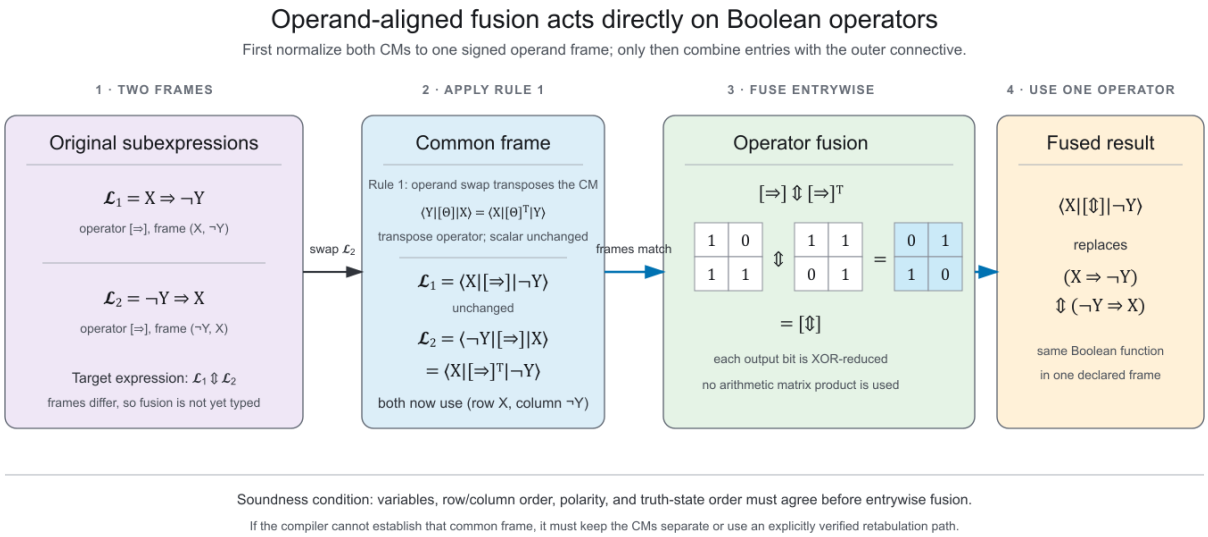


Figure 2: Two operator occurrences are first rewritten into one declared operand frame and are then fused entrywise. The transformed bra–operator–ket expression records the operand exchange explicitly; fusion is not applied while the frames differ.

3.1 Compiler rule and soundness boundary

Fix distinct declared variables R and C in disjoint row and column layout sets. A structural pair result $\mathrm{Pair}(R, C, t)$ is admitted only for a signed binary primitive, the negation of an admitted pair, or the fusion of two admitted children carrying identical (R, C) metadata. Its four-bit token satisfies

$$E_t(r, c) = e[r/R, c/C] \qquad \text{for all } r, c \in \mathbb{B}, \tag{16}$$

after any fixed substitutions, which may not include R or C on a returned pair path. Expressions with one live row variable and one live column variable may instead use an explicitly labeled four-assignment retabulation. Repeated- variable expressions such as $X \,\Theta\, X$, overlapping row/column layouts, compound operands treated as opaque, and different variable pairs are outside the structural rule.

Theorem 1 (Structural alignment-and-fusion soundness)**.** *If the structural rules return* $\mathrm{Pair}(R, C, t)$ *for* e*, then* (16) *holds.*

Proof. Use structural induction over an admitted construction. A signed primitive is correct by (13); complementing a correct child represents negation; and fusing correct children with identical metadata is sound by (15). □

Lemma 1 (Exact pair retabulation)**.** *If the live support after fixed substitution is exactly one distinct row variable* R *and one distinct column variable* C*, evaluating* e *at the four assignments* $(1,1), (1,0), (0,1), (0,0)$ *produces the unique token satisfying* (16)*.*

Theorem 2 (Soundness of the admitted pair compiler)**.** *Whether produced structurally or by exact pair retabulation, every returned* $\mathrm{Pair}(R, C, t)$ *satisfies* (16)*. If neither path is admitted, the ordinary CM IR evaluates the unchanged expression.*

4 Compiler method

4.1 Five artifacts, not one

The implementation contains several objects that can all appear informally as "a CM". Treating them as interchangeable would make both correctness and timing claims ambiguous. Table 1 fixes the terminology used in the remainder of the paper.

Artifact	Meaning	Size/order	Role
CM operator token	One binary truth function packed into four bits	$(11, 10, 01, 00)$; true-first	Constant-size transform and fusion
Pair surrogate	Token plus canonical row and column variable names	One ordered variable pair	Carries the operand-frame invariant
CM IR	Interned Boolean expression DAG	False-first array axes	General symbolic fallback and compilation
Flat program	Instruction sequence compiled from the CM IR	Backend-dependent	Repeated packed or array evaluation
Explicit CM	Dense truth-relation array over declared row/column variables	2^n entries; false-first	User-visible materialized output

Table 1: Implementation artifacts and their declared semantic boundaries. Only the first is the constant-size binary operator of Section 2.

The token is the paper's compact binary operator. The pair surrogate gives that token a frame. The IR and flat program preserve expression structure for more general formulas, whereas an explicit CM materializes the complete requested truth relation. Each benchmark endpoint therefore names the artifact it returns; a token-only endpoint cannot be reported as though it had already paid for an explicit 2^n-entry output.

4.2 Signed-pair implementation

The implementation enforces the disjoint-layout and distinct-variable admissibility conditions stated in Section 3. For a valid request it constructs a result satisfying (16) through the following ordered cases.

1. A supported binary connective over two signed literals is recognized. Finite negation chains determine each literal's polarity. Operand exchange becomes transpose, and negative row or column literals become the corresponding permutations from (13).

2. Negation of an already compiled pair complements the four token bits.

3. Two child pairs are combined only when both canonical row/column names agree. Their outer connective is then applied through a fixed entrywise lookup table, implementing (15).

4. If structural construction stops but the remaining subtree has exactly one live row variable and one live column variable, the compiler evaluates its four assignments and places the results in true-first token order. This is *direct pair retabulation*, reported separately rather than counted as structural fusion.

5. If none of these cases applies, the expression remains on the ordinary CM-IR path.

The supported primitive connectives are AND, OR, XOR, implication, and equivalence. Primitive operands may have any finite stack of negations and may arrive in row/column or column/row order. Recursive pair construction supports negation and the same five connectives when the child metadata match. The structural claim does not cover two arbitrary compound formulas treated as opaque operands, equivalence discovery between compound operands, or fusion across different row/column partitions.

Every recursive pair call receives a strict AST child, so structural compilation terminates for finite expressions. Retabulation performs four finite evaluations. For fixed R, C, and true-first order, the returned four-bit token is canonical for its binary truth function; neither the source syntax nor the surrounding IR is claimed to be globally canonical.

4.3 Layout conversion, ownership, and cost

The compact token and dense array deliberately use different historical orders. Before lifting a token, the implementation reverses both token axes: if T is true-first and D is the false-first 2×2 view, then

$$D_{ab} = T_{1-a,1-b}, \qquad a, b \in \{0, 1\}. \tag{17}$$

Axes absent from the pair are then broadcast into the requested variable layout. The returned dense array is an owned copy; it does not expose a writable broadcast view. Expression nodes and pair surrogates are immutable, tokens are integers, and caller-owned layout and fixed-assignment inputs are read-only under the pair compiler's contract.

Let N be the number of AST occurrences, U the number of unique DAG nodes, H the AST height, S the number of occurrences in a subtree sent to retabulation, D that subtree's evaluation depth, k the length of a signed literal's negation chain, and n the number of requested unfixed Boolean output axes. Inspecting a signed literal costs $O(k)$, followed by an $O(1)$ four-bit alignment transform; token fusion itself is $O(1)$. One retabulation uses $O(4S)$ time and $O(D)$ evaluation-stack space. A fully successful structural pair tree is ordinarily $O(N)$ time and $O(H)$ stack space, although the current support-discovery implementation can rescan nested rejected subtrees and has an $O(N^2)$ worst case. A sharing-aware evaluator may instead scale with U, which is why it is a required comparator. Most importantly, lifting any method to a complete explicit result costs $\Theta(2^n)$ time and output space. Constant-time token fusion is therefore a kernel property, not an end-to-end complexity claim.

The compiler records primitive tokens, signed alignments, token negations, token fusions, direct retabulations, visited nodes, pair attempts, and pair collapses. These counters describe the mechanism and permit workload stratification. They are not substitutes for elapsed time, peak memory, or a comparison with a sharing-aware representation.

4.4 Verification boundary

The current finite suite covers all 16 tokens in all eight signed operand frames, all five primitive connectives in those frames, recursive fusion after alignment, direct retabulation, true-first constants, 250 seeded random two-variable formulas, public token-only behavior, and input nonmutation. The focused pair, optimization, and output-budget suites contain 105 passing tests. Independent exhaustive checks also cover 1,048,576 signed-alignment/fusion cases. These checks strengthen implementation assurance; they do not replace the soundness proof, prove unbounded complexity claims, or establish a speed advantage.

5 Related work and representation boundaries

Several representations considered in this paper carry the same four truth values, but they are not the same mathematical object. The distinction matters both for novelty and for correct use of the operations.

The numeric CM $[\Theta]$ is a 2×2 Boolean operator in a declared row/column frame. Its vectorization

$$\mathrm{vec}([\Theta]) = (\Theta_{11}, \Theta_{10}, \Theta_{01}, \Theta_{00}) \tag{18}$$

is the ordered truth vector, carrying the same four bits in a one-dimensional layout. To make the STP comparison reproducible, adopt the common delta encoding $1 \mapsto \delta_2^1 = [1,0]^\mathsf{T}$ and $0 \mapsto \delta_2^2 = [0,1]^\mathsf{T}$, with Kronecker columns ordered $(11, 10, 01, 00)$. If $t = \mathrm{vec}([\Theta])$, the corresponding binary-output structure matrix is then

$$M_\Theta = \begin{bmatrix} t \\ \neg t \end{bmatrix}. \tag{19}$$

Other source conventions require an explicit row or column permutation [Cheng and Qi, 2010, Zhao et al., n.d.]. The 2×4 STP matrix is an external representation, not a component of a CM.

ANF coefficients are farther removed. For two variables, an algebraic normal form has the shape

$$X \,\Theta\, Y = a_\varnothing \Updownarrow (a_X \wedge X) \Updownarrow (a_Y \wedge Y) \Updownarrow (a_{XY} \wedge X \wedge Y). \tag{20}$$

In the paper's true-first storage order, the Boolean Möbius conversion is unambiguous when written componentwise:

$$a_\varnothing = \Theta_{00}, \qquad a_X = \Theta_{00} \Updownarrow \Theta_{10}, \tag{21}$$

$$a_Y = \Theta_{00} \Updownarrow \Theta_{01}, \qquad a_{XY} = \Theta_{00} \Updownarrow \Theta_{10} \Updownarrow \Theta_{01} \Updownarrow \Theta_{11}. \tag{22}$$

Thus $a_\Theta = (a_\varnothing, a_X, a_Y, a_{XY})$ is a polynomial coordinate representation, not the entries of the formula-valued LM introduced in Section 8. The relevant STP treatment uses Kronecker products of vectors $[1, x_i]^\mathsf{T}$ to enumerate the square-free monomial basis and gives a conversion from truth values to those coefficients [Zhao et al., n.d.]; it does not identify the coefficients with a CM or LM.

Figure 3 summarizes these relationships. Solid boxes on the left are native to the present construction. The truth vector is the explicit bridge to the dashed external comparison boxes; the arrows are declared maps, not claims that all objects have the same type.

This map also separates three senses of linearity. CM evaluation is linear in the operator coefficients over GF(2) for fixed operands (7). ANF calls a Boolean function linear or affine when terms of degree greater than one are absent. Matrix and STP formalisms may additionally be linear in their chosen vector space. None of these statements implies the others without an explicit conversion and its domain.

5.1 Closest operational antecedents

Small truth functions are routinely stored and manipulated as packed bit strings in logic synthesis. In the ABC AIG rewriter, for example, four-input cut functions are stored as 16-bit truth tables, classified up to input negation, input permutation, and output negation (NPN), and matched to precomputed replacement DAGs [Mishchenko et al., 2006]. Modern NPN work likewise defines equivalence by exactly those transformations and uses canonical or semicanonical representatives in synthesis and technology mapping [Zhou et al., 2019]. These precedents mean that CM transposition, row/column permutation, complementation, and packed four-bit storage are not individually new operations.

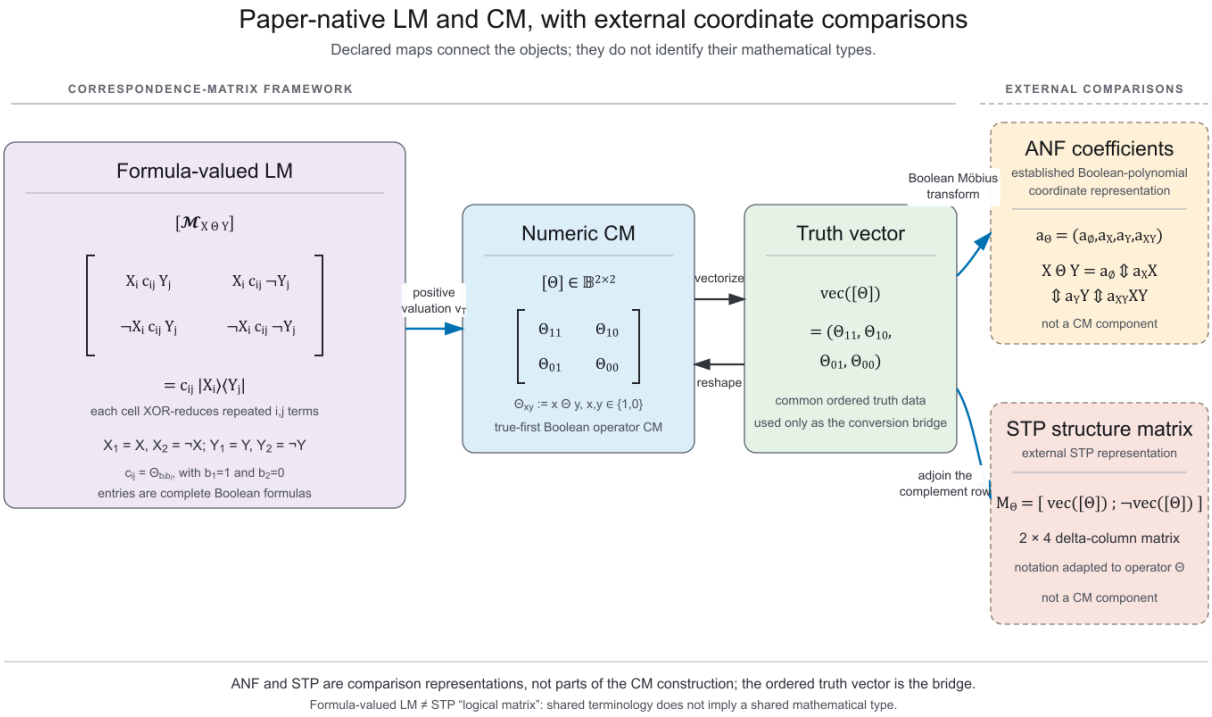


Figure 3: The formula-valued LM is valued into the numeric CM $[\Theta]$. Vectorization exposes the ordered truth values used to compare the CM with ANF coordinates and an STP logical structure matrix. ANF and STP are external representations rather than CM components. The symbols a_Θ and M_Θ are local, operator-indexed labels; source-specific notation varies.

Logic-synthesis practice also supplies the strongest comparison for the compiler claim. AIG rewriting preserves and exploits DAG sharing rather than only the final truth function [Mishchenko et al., 2006]; broader synthesis flows move among truth tables, graph networks, and exact or heuristic optimization according to support size and task [Testa et al., 2019]. Bit-parallel simulation packs many assignments into machine words and evaluates a network over those patterns at once [Lee et al., 2022]. The present compiler therefore does not claim a new NPN classifier, LUT mapper, AIG rewriter, or bit-parallel evaluator. Its candidate contribution is narrower: an explicitly typed two-operand frame used as an admission certificate for local operator fusion, with a measured fallback boundary.

Family	Operator/data shape	Scalar algebra	Transform or composition role	Relation to this work
Stern/Mizraji matrix logic	Truth vectors, tensor inputs, matrix gates	Primarily ordinary real/vector algebra	Logical operators and operator bases	Establishes the operator viewpoint and outer-product antecedents
Cheng/Zhao STP and Boolean matrices	2×2^n structure matrices and Boolean matrices	STP plus explicit XOR–AND Boolean products	Swap/negation matrices and pointwise truth-function combination	Direct overlap with Boolean matrix algebra and fusion identities
ANF	Square-free monomial coefficient vector	GF(2) polynomial algebra	Möbius coordinate conversion	Different map and type from a formula-valued LM
NPN/LUT/AIG synthesis	Packed truth tables and shared Boolean DAGs	Bit operations and graph rewrites	Input/output phase, permutation, cut matching, and shared replacement	Closest operational prior art; mandatory baseline family
CM pair compiler	Four-bit token plus declared row/column frame	XOR–AND contraction and entrywise outer connective	Typed signed-frame alignment followed by admitted local fusion	Candidate contribution is the typed integration and measured boundary

Table 2: Claim boundary against the closest representation and compiler families. No row alone establishes novelty; the empirical question concerns the integrated typed CM compiler path.

6 Experimental method

The confirmatory evaluation is designed around a conditional question: where, if anywhere, does operand-aligned token compilation reduce user-visible cost? All arms consume the same immutable AST and declared variable order. The structural-pair arm is compared with (i) a retabulation-only ablation that derives the token by four scalar evaluations, (ii) the ordinary CM IR under the same materialization policy, (iii) a direct sharing-aware packed evaluator of the source AST, (iv) a lowered sharing-aware packed program for prepared reuse, (v) an ROBDD with declared variable order [Bryant, 1986], and, for translatable circuit workloads, (vi) an external AIG flow. A four-bit LUT query is also reported after both the packed evaluator and CM compiler have produced the same truth token. NPN canonicalization is measured only where it is actually invoked by the synthesis baseline; it is not charged to CM or presented as an evaluation algorithm. Backend-native preprocessing is allowed but is timed and reported.

For the direct two-variable packed arm, assignment bit positions are $(00, 01, 10, 11)$ from least to most significant. The row and column masks are therefore 1100_2 and 1010_2, respectively. One memoized traversal applies the source connectives as machine-bit operations and returns a four-bit truth word; read most-significant bit first, that word is exactly the CM token order $(11, 10, 01, 00)$. This baseline does not construct a 2×2 array, perform four separate scalar traversals, or use CM alignment rules. It is consequently the closest direct test of whether structural pair recognition saves work over ordinary bit-parallel truth-table evaluation [Lee et al., 2022].

6.1 Workloads and endpoints

The generated corpus separates exact-frame formulas, signed or swapped frames, two-variable retabulation cases, formulas involving multiple operand pairs, compound-operand cases outside the structural claim, and untouched natural formulas or circuits. Generated strata cross AST sizes $15, 63, 255, 1023, 4095$, balanced and skewed tree shapes, all five supported outer connectives, and reuse counts $1, 10, 100, 1000$. Requested outputs include the compact token, scalar evaluation, packed batches, and explicit dense CMs where the output budget permits them.

The primary endpoints are one-shot end-to-end time, prepared repeated- evaluation time, and peak process memory above a pre-run baseline. Each arm uses a named timing boundary: structural recognition plus token construction; four complete scalar traversals for retabulation; one packed

AST traversal; lowering plus packed-program execution; token/LUT query after preparation; and conversion plus allocation for any explicit dense result. Preparation and query time are never pooled silently. In particular, once either method has the same four-bit token, its LUT query is identical; any advantage must arise before that common endpoint or from a measured reuse policy.

Endpoints are reported separately by workload stratum, output type, and reuse count. Mechanism diagnostics include token-only compilation, support discovery, conversion and allocation, all pair counters, representation size, explicit output bytes, exceptions, timeouts, and budget rejections. Break-even reuse is computed from measured preparation and per-evaluation costs rather than extrapolated from the four-bit kernel alone.

6.2 Correctness and sampling

Correctness is checked before performance results are summarized. Completed arms must agree with an independent truth-table oracle whenever exhaustive evaluation fits the output budget; larger cases use a frozen random assignment set and an independent equivalence check where available. Inputs must remain unchanged. Every disagreement, exception, timeout, or rejection remains in the raw results with its case identifier and disposition.

Measurements run in isolated processes on a recorded software and hardware environment. Correctness and one untimed warm-up precede sampling. Arm order is randomized within each case using a recorded schedule seed. Generated cells receive at least 30 valid repetitions and natural cases at least 15 unless the predeclared 60-second timeout intervenes. Timed inner loops last at least 100 milliseconds when repeated evaluation is semantically appropriate. Peak memory is measured in a fresh process without subtracting undeclared caches.

For paired comparisons, the analysis reports median absolute time, competitor- to-CM time ratio, median memory difference, case-level distributions, and a 95% hierarchical bootstrap interval resampling cases and repetitions. A practical positive region requires zero correctness disagreements; at least a 20% median structural-over-retabulation improvement in the exact and signed pair strata with the interval above one; and at least one predeclared cell with the same end-to-end improvement over a sharing-aware external baseline without more than a 10% median peak-memory increase. Falling below that rule narrows the result; it does not justify selecting a more favorable endpoint after the fact.

The protocol, source state, corpus, seeds, arm configuration, schedule, environment, test output, and append-only raw observations are content-hashed before the first confirmatory run. Any substantive change produces a new protocol version. This freeze keeps the empirical claim separate from the examples used to develop the compiler.

7 Results and evidence status

The formal and implementation evidence currently clears the following pre-performance gates: exhaustive token and signed-frame checks, exhaustive signed alignment/fusion enumeration, randomized two-variable differential checks, and public-API tests for structural, retabulation, fallback, output, and input-ownership behavior. These results support correctness of the tested implementation surface; they do not establish a runtime or memory advantage.

No confirmatory performance table or Figure 5 appears in this working draft. The corpus and run manifest have not yet been frozen and the campaign has not been executed. Quantitative speed, memory, break-even, prevalence, and failure- region claims therefore remain open. Kernel-only timings and previously superseded headline speedups will not be reused as end-to-end results. This is an explicit evidence boundary rather than a missing positive result: the final section will report

every frozen arm and stratum, including ties, losses, timeouts, and cases in which pair recognition falls back.

8 Formula-valued logical matrices

8.1 Symbolic lift and positive valuation

Let $\mathcal{F}$ be the Boolean algebra of formulas modulo logical equivalence. Define the polarity orbit

$$X_1 = X, \quad X_2 = \neg X, \qquad Y_1 = Y, \quad Y_2 = \neg Y. \tag{23}$$

Let $b_1 = 1$ and $b_2 = 0$, and reserve Θ_{xy} for truth assignments $x, y \in \mathbb{B}$. Define the position-indexed LM coefficients by

$$c_{ij} := \Theta_{b_i b_j}, \qquad i, j \in \{1, 2\}. \tag{24}$$

Thus $(c_{11}, c_{12}, c_{21}, c_{22})$ denotes the same four values as $(\Theta_{11}, \Theta_{10}, \Theta_{01}, \Theta_{00})$ without overloading one subscript system. For Boolean coefficients c_{ij}, define the formula-valued logical matrix by the XOR sum of its four polarity outer products:

$$[\mathcal{M}_{X\Theta Y}] := \Updownarrow_{i,j\in\{1,2\}} c_{ij}\ |X_i\rangle\langle Y_j|\,. \tag{25}$$

Here $|X_i\rangle\langle Y_j|$ is the formula-valued outer product

$$|X_i\rangle\langle Y_j| = \begin{bmatrix} X_i \wedge Y_j & X_i \wedge \neg Y_j \\ \neg X_i \wedge Y_j & \neg X_i \wedge \neg Y_j \end{bmatrix}. \tag{26}$$

Multiplication by c_{ij} means entrywise conjunction. Expanding the XOR sum gives

$$[\mathcal{M}_{X\Theta Y}] = \begin{bmatrix} \Updownarrow_{i,j} X_i \wedge c_{ij} \wedge Y_j & \Updownarrow_{i,j} X_i \wedge c_{ij} \wedge \neg Y_j \\ \Updownarrow_{i,j} \neg X_i \wedge c_{ij} \wedge Y_j & \Updownarrow_{i,j} \neg X_i \wedge c_{ij} \wedge \neg Y_j \end{bmatrix} = \begin{bmatrix} X\ \Theta\ Y & X\ \Theta\ \neg Y \\ \neg X\ \Theta\ Y & \neg X\ \Theta\ \neg Y \end{bmatrix}. \tag{27}$$

Thus every cell in (27) is itself a complete XOR-reduced formula; the four displayed cells are not four separate terms of one formula. The entries lie in $\mathcal{F}$, not in a numeric vector space. We use *formula-valued LM* to distinguish this object from the numeric "logical matrix" and logical structure matrix of the STP literature.

A valuation $v : \mathcal{F} \to \mathbb{B}$ acts entrywise and preserves $\neg$, $\wedge$, and $\Updownarrow$. Let v_T be the positive valuation with $v_T(X) = v_T(Y) = 1$. Then

$$v_T([\mathcal{M}_{X\Theta Y}]) = [\Theta]. \tag{28}$$

Under a general valuation $v(X) = x$ and $v(Y) = y$, the result is the same CM with the row and column permutations determined by x and y. If $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, then

$$v([\mathcal{M}_{X\Theta Y}]) = P^{1-x}[\Theta]P^{1-y}. \tag{29}$$

The LM is therefore a symbolic polarity-orbit matrix whose valuations produce numeric CMs in declared operand frames. It is not an ANF coefficient vector: positive valuation, rather than a Möbius transform, is the map from this LM to $[\Theta]$.

8.2 Logical pairing

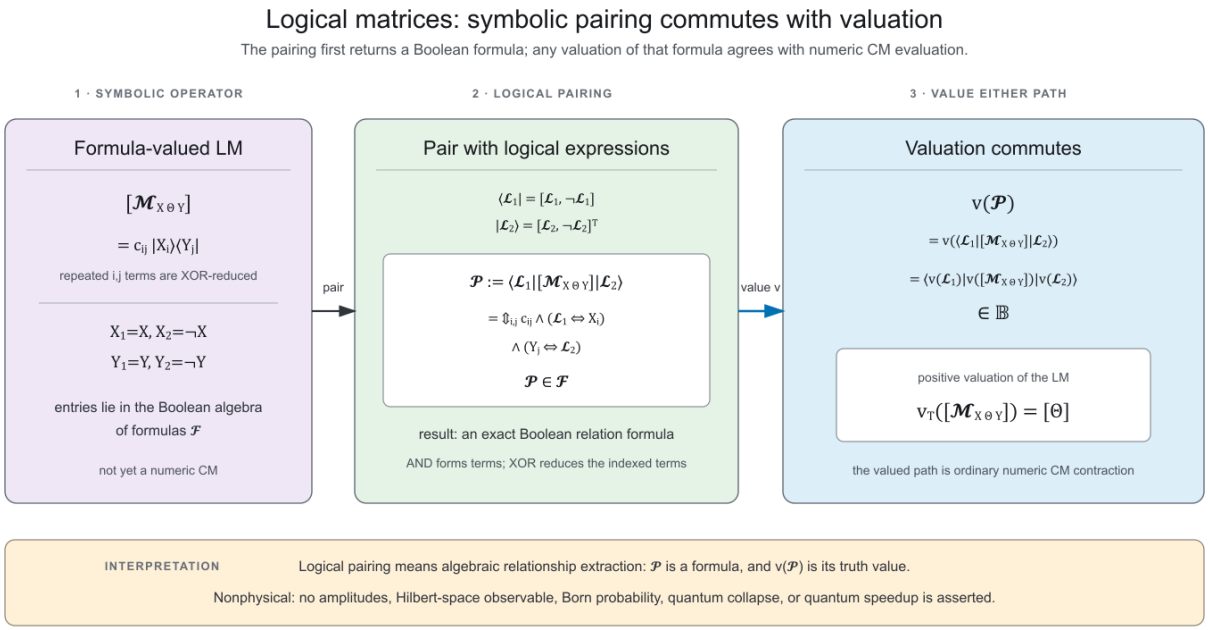


Figure 4: A formula-valued LM is paired with logical expressions before or after valuation. Because valuation is a Boolean-algebra homomorphism, both paths give the same truth value. Logical pairing denotes algebraic relationship extraction, not a physical measurement process.

For a formula $A \in \mathcal{F}$, define

$$|A\rangle = \begin{bmatrix} A \\ \neg A \end{bmatrix}, \qquad \langle A| = \begin{bmatrix} A & \neg A \end{bmatrix}. \tag{30}$$

The XOR–AND contraction of two such states gives logical equivalence:

$$\langle A|\,|B\rangle = (A \wedge B) \,\Updownarrow\, (\neg A \wedge \neg B) = (A \Leftrightarrow B). \tag{31}$$

For arbitrary formulas $\mathcal{L}_1$ and $\mathcal{L}_2$, define the logical pairing

$$\mathcal{P} := \langle \mathcal{L}_1|\,[\mathcal{M}_{X\Theta Y}]\,|\mathcal{L}_2\rangle \tag{32}$$

$$= \Updownarrow_{i,j\in\{1,2\}} c_{ij} \wedge (\mathcal{L}_1 \Leftrightarrow X_i) \wedge (Y_j \Leftrightarrow \mathcal{L}_2). \tag{33}$$

The output $\mathcal{P} \in \mathcal{F}$ has an exact semantic form.

Proposition 2 (Logical-pairing identity)**.** *For every $A, B \in \mathcal{F}$,*

$$\langle A|\,[\mathcal{M}_{X\Theta Y}]\,|B\rangle \equiv (A \Leftrightarrow X)\,\Theta\,(Y \Leftrightarrow B). \tag{34}$$

For example, setting $A = X$ and $B = Y$ makes both equivalence factors true, so the pairing evaluates the declared coefficient $1\,\Theta\,1 = \Theta_{11}$. More generally, the equivalence factors state which polarities of the two operands are presented to Θ; the full proof is in the appendix.

Theorem 3 (Valuation commutes with LM pairing)**.** *For every Boolean-algebra valuation v,*

$$v(\mathcal{P}) = \langle v(\mathcal{L}_1)|\, v([\mathcal{M}_{X\Theta Y}])\, |v(\mathcal{L}_2)\rangle\,. \tag{35}$$

Proof. The finite pairing in (33) is built only from negation, conjunction, and exclusive-or. A Boolean valuation preserves each of these operations, so it can be moved through every term and through the finite XOR reduction. □

The primary term in this paper is *logical pairing*. The historical word *measurement* refers only to extracting a coefficient or relationship from a symbolic operator. No amplitudes, Hilbert-space observable, Born probability, physical state collapse, entanglement, or quantum speedup is asserted. Operator and bra–ket notation also appears in explicitly quantum-adjacent logical formalisms such as Eigenlogic [Toffano, 2015, Dubois and Toffano, 2016]; those formalisms include mathematical structures that are absent here. Figure 4 makes the nonphysical interpretation explicit.

9 Discussion and limitations

The explicit representation of an arbitrary n-variable Boolean function contains 2^n truth values. It may be written as an order-n tensor of side two, a length-2^n vector, or, after an even split of $2m$ variables, a $2^m \times 2^m$ matrix. No reshape removes this worst-case output size. Compression and symbolic sharing must therefore be evaluated separately.

Operator fusion also depends on repeated, alignable operands. When compatible pairs are rare, when normalization dominates, or when another representation preserves substantially more sharing, the CM path may tie or lose. These are expected boundary cases, not failures to be hidden. The final discussion will interpret them together with preparation cost, memory traffic, reuse count, variable ordering, and fallback frequency.

10 Conclusion

Correspondence matrices are best understood as typed Boolean operators rather than as a novelty claim about reshaped truth tables. XOR–AND contraction lets one-hot operand states evaluate those operators; matrix transformations align signed operand frames; and exact compatible frames can be fused before evaluation. Formula-valued logical matrices add a symbolic layer whose valuation recovers numeric CMs and whose logical pairing remains within Boolean algebra. The remaining empirical question is deliberately practical: which workloads permit this operator-level rewrite to reduce total computation after all costs are counted?

A Proof details

This appendix collects the typed arguments supporting the concise statements in the main text. All scalar sums are XOR reductions and all scalar products are conjunctions unless another algebra is named explicitly.

A.1 Representation and selection

Proposition 3 (Unique binary representation)**.** *The map from a binary Boolean operator Θ to $[\Theta]$ is a bijection between functions $\mathbb{B}^2 \to \mathbb{B}$ and matrices in $\mathbb{B}^{2\times 2}$ once the row, column, and truth-state order is fixed.*

Proof. A binary Boolean function is determined by its four values on $(1,1),(1,0),(0,1),(0,0)$. These are exactly the four entries of $[\Theta]$. Conversely, any four-bit matrix assigns one output to each binary input pair and therefore defines one such function. Both sets contain $2^4 = 16$ elements. □

For completeness, expand the contraction as

$$E_{[\Theta]}(x,y) = \Updownarrow_{r,s\in\{1,0\}} \chi_x(r) \wedge \Theta_{rs} \wedge \chi_y(s), \qquad \chi_z(q) := \begin{cases} 1, & q = z, \\ 0, & q \neq z. \end{cases} \tag{36}$$

The two one-hot states make $\chi_x(r) \wedge \chi_y(s)$ equal to one only at $(r,s) = (x,y)$. This proves selection without a cancellation or distributivity assumption beyond the declared Boolean algebra. It also explains why OR–AND selection would return the same scalar on one-hot states, although OR does not supply the same coefficient-space algebra for arbitrary vectors.

A.2 Coefficient linearity and basis

For fixed x, y, distribute conjunction over XOR:

$$E_{A \Updownarrow B}(x,y) = \Updownarrow_{r,s} \chi_x(r) \wedge (A_{rs} \Updownarrow B_{rs}) \wedge \chi_y(s) \tag{37}$$

$$= E_A(x,y) \Updownarrow E_B(x,y). \tag{38}$$

Thus evaluation is a coordinate projection and hence a linear functional on $\mathrm{GF}(2)^{2\times 2}$. Let E_{rs} be the matrix with a one only at position (r,s). Evaluating

$$[\Theta] = \Updownarrow_{r,s} \Theta_{rs} E_{rs} \tag{39}$$

at any position selects Θ_{rs}, proving the basis decomposition and coefficient extraction. This proof concerns the coefficient matrix. The affine truth-state encoding prevents it from implying that every represented Boolean function is linear in its logical inputs.

A.3 Signed operand transformations

Let $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. The row-$x$, column-$y$ entry of $P[\Theta]$ is the row-$\neg x$, column-y entry of $[\Theta]$ and is therefore $(\neg x)\,\Theta\, y$. Similarly, right multiplication by P exchanges the column assignment and transposition exchanges the two input assignments. Hence

$$[\Theta(Y,X)] = [\Theta]^{\mathsf{T}}, \qquad [\Theta(\neg X,Y)] = P[\Theta], \tag{40}$$

$$[\Theta(X,\neg Y)] = [\Theta]P, \qquad [\Theta(\neg X,\neg Y)] = P[\Theta]P. \tag{41}$$

Negating every output flips every bit, which is entrywise XOR with the all-ones matrix. Products with P are XOR–AND products, but only one entry survives in each output position, so they can equivalently be implemented as direct permutations.

For XNOR, $[\Leftrightarrow]$ is the identity matrix. Under the explicit clockwise convention $\mathrm{rot}_{90^\circ}\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} c & a \\ d & b \end{bmatrix}$, its ones move to the antidiagonal, giving $[\Updownarrow]$. (For this particular symmetric matrix, counterclockwise rotation gives the same result.) The two supports are disjoint and cover all four positions, so their entrywise XOR is the all-ones $[\Diamond]$.

A.4 Operand-aligned fusion

Let Φ be any binary Boolean operator and define

$$F_{rs} := (\Theta_1)_{rs}\, \Phi\, (\Theta_2)_{rs}. \tag{42}$$

When both inner operators use the same operand frame, selection gives

$$(x\, \Theta_1\, y)\, \Phi\, (x\, \Theta_2\, y) \tag{43}$$

$$= (\Theta_1)_{xy}\, \Phi\, (\Theta_2)_{xy} = F_{xy} = \langle x|\, F\, |y\rangle\,. \tag{44}$$

No general distributive law for Φ over XOR is used. If the original operators differ by signed operand order, the preceding transformations first translate each one into the target frame; applying the same argument after that translation proves fusion after alignment.

For structural compilation, proceed by induction on the admitted expression tree. A primitive signed-literal pair is sound by the transformations above. For the unary step, complementing all four token bits implements logical negation. For the binary step, the induction hypotheses give two sound child pairs in the same typed frame, and the preceding entry-selection calculation proves their entrywise outer-connective fusion. Every recursive call visits a strict AST child, proving termination. This establishes the structural soundness theorem in Section 3 without using retabulation.

For exact pair retabulation, evaluate the subtree independently at $(1,1),(1,0),(0,1),(0,0)$ and store the four observed bits in that order. Selection at any assignment returns the stored value by construction. The top-level compiler is therefore sound because it returns only a structurally proved pair, an exactly retabulated pair, or the ordinary IR result. The ordinary IR retains responsibility for every expression not admitted by the first two cases.

A.5 Formula-valued LMs

Let $B_{ij}(X,Y) = |X_i\rangle\langle Y_j|$. The upper-left cell of

$$[\mathcal{M}_{X\Theta Y}] = \Updownarrow_{i,j\in\{1,2\}} c_{ij} B_{ij}(X,Y), \qquad c_{ij} := \Theta_{b_i b_j}, \quad b_1 = 1,\ b_2 = 0, \tag{45}$$

is the disjoint-minterm expansion

$$c_{11} XY \Updownarrow c_{12} X\neg Y \Updownarrow c_{21} \neg XY \Updownarrow c_{22} \neg X\neg Y. \tag{46}$$

Under the declared position-to-assignment map, this is $X\,\Theta\,Y$. Exchanging the Y components gives $X\,\Theta\,\neg Y$; exchanging the X components gives $\neg X\,\Theta\,Y$; and exchanging both gives the lower-right formula. This proves the polarity-orbit form in (27).

If $v(X) = x$ and $v(Y) = y$, applying v to those four formulas either preserves or exchanges the two row and column assignments. It follows that

$$v([\mathcal{M}_{X\Theta Y}]) = P^{1-x}[\Theta]P^{1-y}. \tag{47}$$

The positive case $x = y = 1$ yields $[\Theta]$.

For formulas A, B,

$$\langle A|\,|B\rangle = (A \wedge B) \Updownarrow (\neg A \wedge \neg B) = (A \Leftrightarrow B), \tag{48}$$

because the two conjunctions are disjoint. Substituting the LM basis expansion between $\langle \mathcal{L}_1|$ and $|\mathcal{L}_2\rangle$ and using this identity on both sides of each outer product gives

$$\Updownarrow_{i,j} c_{ij} \wedge (\mathcal{L}_1 \Leftrightarrow X_i) \wedge (Y_j \Leftrightarrow \mathcal{L}_2). \tag{49}$$

For each valuation of A, B, X, Y, exactly one i satisfies $A \Leftrightarrow X_i$ and exactly one j satisfies $Y_j \Leftrightarrow B$. If these positions are i^*, j^*, the XOR therefore selects $c_{i^*j^*} = \Theta_{b_{i^*} b_{j^*}}$. By the definitions of the polarity orbit and b_i, $b_{i^*} = A \Leftrightarrow X$ and $b_{j^*} = Y \Leftrightarrow B$. Hence the selected coefficient is $(A \Leftrightarrow X)\,\Theta\,(Y \Leftrightarrow B)$, proving (34).

Finally, a Boolean valuation is a homomorphism for negation, conjunction, and XOR. Moving it through this finite expression term by term proves (35).

A.6 ANF conversion is a different map

For n variables, identify a valuation with its support $S \subseteq \{1, \ldots, n\}$ and let t_S be the corresponding truth value. If

$$f(x_1, \ldots, x_n) = \Updownarrow_{T \subseteq \{1,\ldots,n\}} a_T \bigwedge_{i \in T} x_i, \tag{50}$$

then evaluation on support S gives

$$t_S = \Updownarrow_{T \subseteq S} a_T. \tag{51}$$

Boolean Möbius inversion gives the converse relation

$$a_T = \Updownarrow_{S \subseteq T} t_S. \tag{52}$$

Thus ANF coefficients arise by an invertible coordinate transform of the truth vector. By contrast, positive valuation maps the formula-valued polarity orbit directly to a numeric CM. The two maps have different domains and purposes even though both ultimately describe the same Boolean function.

A.7 Higher-arity size

An arbitrary function of n Boolean variables has one output for each of 2^n valuations. Its explicit operator therefore has 2^n bits. The same data can be placed in an order-n tensor with side length two, a length-2^n vector, or, after splitting $2m$ variables into equal row and column groups, a $2^m \times 2^m$ matrix. These are layout choices with the same entry count, not compression results.

B Finite verification scope

The accompanying checker exhaustively verifies the 16 binary tokens, 16,384 same-frame fusion cases, 512 signed alignments, 1,048,576 signed alignment/fusion cases, the XOR/XNOR/Impax identities, LM positive and general valuations, 256 LM coefficient extractions, 16,384 LM valuation/fusion cases, and 64 two-variable ANF round trips. Finite enumeration catches convention and implementation errors but is not used as a proof of an unbounded theorem or a runtime bound.

References

Randal E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986. doi: 10.1109/TC.1986.1676819.

Daizhan Cheng and Hongsheng Qi. A linear representation of dynamics of boolean networks. *IEEE Transactions on Automatic Control*, 55(10):2251–2258, 2010. doi: 10.1109/TAC.2010.2043294.

Daizhan Cheng, Yin Zhao, and Xiangru Xu. Matrix approach to boolean calculus. In *Proceedings of the 50th IEEE Conference on Decision and Control and European Control Conference*, pages 6950–6955, 2011. doi: 10.1109/CDC.2011.6160289.

Brian Droncheff. Correspondence matrices; algorithms for propositional logic, May 2018. URL `https://doi.org/10.13140/RG.2.2.28036.37764`. ResearchGate preprint, version 1.0.1.

François Dubois and Zeno Toffano. Eigenlogic: A quantum view for multiple-valued and fuzzy systems, 2016. URL `https://arxiv.org/abs/1607.03509`.

C. R. Edwards. The logic of boolean matrices. *The Computer Journal*, 15(3):247–253, 1972. doi: 10.1093/comjnl/15.3.247.

Winston Lee, Heinz Riener, Alan Mishchenko, Robert K. Brayton, and Giovanni De Micheli. A simulation-guided paradigm for logic synthesis and verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(8):2573–2586, 2022. doi: 10.1109/TCAD.2021.3108704.

Alan Mishchenko, Satrajit Chatterjee, and Robert K. Brayton. DAG-aware AIG rewriting: A fresh look at combinational logic synthesis. In *Proceedings of the 43rd Annual Design Automation Conference*, pages 532–536, 2006. doi: 10.1145/1146909.1147048.

Eduardo Mizraji. Vector logics: The matrix–vector representation of logical calculus. *Fuzzy Sets and Systems*, 50(2):179–185, 1992. doi: 10.1016/0165-0114(92)90216-Q.

Eduardo Mizraji. Vector logic: A natural algebraic representation of the fundamental logical gates. *Journal of Logic and Computation*, 18(1):97–121, 2008. doi: 10.1093/logcom/exm057.

August Stern. *Matrix Logic: Theory and Applications*. North-Holland, 1988. doi: 10.1016/C2009-0-14445-8.

Eleonora Testa, Mathias Soeken, Luca Gaetano Amarù, and Giovanni De Micheli. Logic synthesis for established and emerging computing. *Proceedings of the IEEE*, 107(1):165–184, 2019. doi: 10.1109/JPROC.2018.2869760.

Zeno Toffano. Eigenlogic in the spirit of george boole, 2015. URL `https://arxiv.org/abs/1512.06632`.

Yin Zhao, Xu Gao, and Daizhan Cheng. Semi-tensor product approach to boolean functions. Unpublished preprint, n.d. URL `https://lsc.amss.ac.cn/~dcheng/preprint/bf01.pdf`.

Xuegong Zhou, Lingli Wang, and Alan Mishchenko. Fast adjustable NPN classification using generalized symmetries. *ACM Transactions on Reconfigurable Technology and Systems*, 12(2):7, 2019. doi: 10.1145/3313917.